Прогнозирование продаж Corporación Favorita

Курс «Анализ временных рядов», НИУ ВШЭ, 2025–2026

Команда: Степан Панкратов, Максим Сидоров

---

1. Постановка задачи

Предсказать ежедневные продажи товаров в сети продуктовых магазинов Эквадора на горизонт **16 дней** вперёд на основе датасета Kaggle “Corporación Favorita Grocery Sales Forecasting”.

Метрика — Normalized Weighted Root Mean Squared Logarithmic Error (NWRMSLE):

$$\mathrm{NWRMSLE} = \sqrt{\frac{\sum_i w_i \big(\ln(1+\hat{y}_i) - \ln(1+y_i)\big)^2}{\sum_i w_i}}$$

где $w_i = 1.25$ для скоропортящихся товаров (`perishable`) и $w_i = 1.0$ для остальных.

Такая метрика хорошо подходит задаче. Во-первых, логарифм сглаживает влияние очень больших продаж и делает сравнение рядов более устойчивым. Во-вторых, повышенный вес скоропортящихся товаров отражает реальную бизнес-логику: ошибка на свежей продукции обходится дороже, чем на товарах с длинным сроком хранения.

Масштаб данных: `train.csv` содержит около **125 млн строк**. Это продажи примерно **4000 товаров** в **54 магазинах** за **4.5 года**. Тестовый набор покрывает следующие 16 дней после конца train.

---

2. Команда и распределение работы

Участник	Зона ответственности
Степан Панкратов	Сборка, отладка основного пайплайна: загрузка данных чанками, preprocessing, feature engineering, обучение CatBoost и MIMO MLP, сбор ансамбля и генерация финального submission
Максим Сидоров	EDA, визуализации и проверка сезонных паттернов, анализ ошибок по `family` / `store` / `store type`, ин-

Участник	Зона ответственности
	терпретация результатов, подготовка ноутбука и подготовка итогового отчёта

Решения по схеме валидации, списку признаков и выбору финальной модели мы обсуждали совместно.

---

3. Описание данных

Файл	Описание	Ключевые поля
train.csv	История продаж (125M строк)	`date`, `store_nbr`, `item_nbr`, `unit_sales`, `onpromotion`
test.csv	16 дней для предсказания	`id`, `date`, `store_nbr`, `item_nbr`, `onpromotion`
stores.csv	Метаданные магазинов (54 шт.)	`store_nbr`, `city`, `state`, `type`, `cluster`
items.csv	Метаданные товаров (~4000 шт.)	`item_nbr`, `family`, `class`, `perishable`
oil.csv	Цена нефти как внешний фактор	`date`, `dcoilwtico`
holidays_events.csv	Праздники и специальные даты	`date`, `type`, `locale`, `locale_name`, `transferred`
transactions.csv	Число транзакций по магазинам	`date`, `store_nbr`, `transactions`

Что пришлось учитывать при подготовке данных: - Отрицательные значения `unit_sales` это возвраты, поэтому мы заменяли их на `0` - В `oil.csv` есть пропуски на выходных, поэтому цену нефти восстанавливали через forward-fill - В ранней части train поле `onpromotion` часто пустое, такие значения мы заполняли нулём - Данные заметно разрежены: существенная доля наблюдений приходится на нулевые продажи

Именно сочетание большого объёма, разреженности и большого числа товарных рядов делает задачу нетривиальной даже для довольно сильных табличных моделей.

---

4. Протокол валидации

Мы использовали **фиксированный temporal split**: - из `train.csv` берем **66 дней** (`2017-06-11` — `2017-08-15`) - последние **16 дней** этого окна (`2017-07-31` — `2017-08-15`) используемя как validation - предыдущие 50 дней идут на обучение

Мы специально не делали многократный backtesting через `TimeSeriesSplit`. Так как для этой задачи важнее качественно смоделировать один реальный сценарий: есть история до текущего дня и нужно один раз предсказать следующие 16 дней. И на таком объёме данных несколько fold-ов сильно увеличивают время расчёта, особенно если каждый запуск заново строит лаги и rolling-признаки.

Для экономии памяти мы использовали **sparse grid**. Вместо полного декартова произведения `даты × (store, item)` брались только реально существующие наблюдения из train. На практике это заметно уменьшило размер таблицы признаков и позволило удержать расчёты в разумных пределах по памяти и времени.

---

5. Методология (пайплайн)

Общий пайплайн выглядел так:

Загрузка данных → Preprocessing → Feature Engineering → Обучение моделей → Валидация → Ensemble → Submission

Основные шаги: 1. **Загрузка**: `train.csv` читается чанками по `2M` строк и сразу фильтруется до нужного временного окна 2. **Preprocessing**: строится рабочий grid, очищаются продажи, формируется целевая переменная $target = \log(1 + sales)$ 3. **Feature engineering**: считаются лаги, rolling-статистики, промо-признаки, календарные и внешние признаки 4. **Обучение моделей**: статистические бейзлайны, CatBoost и MIMO MLP 5. **Финальный этап**: подбирается вес ансамбля и собирается submission для Kaggle

Всего в модели использовалось около **44 признаков**: **35 числовых** и **9 категориальных**. Главная идея была не в экзотической архитектуре, а в том, чтобы аккуратно собрать информативные признаки на нескольких уровнях агрегации: `store × item`, `item`, `store`, `store × family`.

---

6. Описание моделей

6.1 Статистические бейзлайны

В качестве отправной точки мы реализовали четыре простых подхода через `StatsForecast`: - **Naive**: прогноз равен последнему наблюдению - **SeasonalNaive(7)**: значение берётся с лагом в 7 дней - **AutoTheta(7)**: автоматический подбор Theta-модели с недельной сезонностью - **AutoETS(7)**: экспоненциальное сглаживание с автоматическим подбором компонент

Бейзлайны считались на случайной подвыборке **1000 пар (`store, item`)**, потому что полный прогон на всех рядах был бы слишком тяжёлым и мало что добавил бы для сравнения базовых идей.

6.2 CatBoost

Основная модель проекта — **CatBoost**, который предсказывает $target = \log(1 + sales)$ для каждой строки grid напрямую. Мы не добавляли горизонт прогноза как отдельный признак: модель учится на общей таблице признаков, а на тесте получает snapshot признаков на последней доступной дате.

Этот подход оказался особенно удачным по двум причинам: - CatBoost хорошо работает с разнородными табличными признаками - модель умеет нативно обрабатывать категориальные поля вроде `store_nbr`, `item_nbr`, `family`, `type`, что избавляет от громоздкого one-hot кодирования

Наиболее важные признаки: - `item_nbr`, `store_nbr` - `lag_7`, `lag_14` - `sales_mean_7d`, `sales_mean_14d`, `sales_mean_30d` - `promo_mean_7d`, `onpromotion` - `day_of_week`

Гиперпараметры: `iterations=1000`, `learning_rate=0.05`, `depth=8`, `loss=RMSE`, early stopping примерно на 480-й итерации.

На тесте для каждого дня прогноза мы использовали признаки, посчитанные на последнем дне train, а значение `onpromotion` брали из `test.csv`. Это не идеальная рекурсивная схема, но она проста, устойчива и достаточно хорошо работает на этой задаче.

6.3 MIMO MLP (Multi-Input Multi-Output)

Нейросетевая модель строилась как **MIMO MLP**, то есть один forward pass сразу выдаёт прогноз на все **16 дней** горизонта.

Архитектура:

```
[числовые фичи] ⊕ [embeddings категориальных] → Linear(512) → ReLU → Dropout(0.2)
    → Linear(256) → ReLU → Dropout(0.2) → Linear(128) → ReLU → Linear(16)
```

Для категориальных признаков использовались embedding-слои, чтобы не раздувать размерность входа. Целевые значения для горизонтов `1..16` строились через `groupby().shift(-h)`.

Важно, что NN обучалась не на полном датасете, а на подвыборке **5000 пар (`store, item`)**. Это было техническим компромиссом: модель хотелось попробовать, но полный запуск в исходной постановке требовал бы более агрессивной оптимизации памяти и времени.

6.4 Ансамбль

Финальный ансамбль строился как линейное смешивание:

$$\hat{y} = \alpha \hat{y}_{\text{CatBoost}} + (1 - \alpha) \hat{y}_{\text{NN}}$$

Параметр `alpha` подбирался grid search на validation в диапазоне от `0.1` до `0.9` с шагом `0.1`. Лучшим оказался вариант `alpha = 0.8`, то есть ансамбль и так в основном опирался на CatBoost.

---

7. Результаты

7.1 Сравнение моделей

Модель	Val NWRMSLE	Val WMAPE
Naive	0.7451	0.7578
SeasonalNaive(7)	0.7103	0.6001
AutoTheta(7)	0.6319	0.5991
AutoETS(7)	0.6445	0.5448
MIMO MLP	0.5690	—
CatBoost	**0.5048**	**0.4438**
Ensemble (α=0.8)	0.5155	—

Лучший результат на локальной валидации показал **CatBoost**: `NWRMSLE = 0.5048`. Это примерно на **20% лучше**, чем лучший статистический бейзлайн `AutoTheta (0.6319)`, и примерно на **11% лучше**, чем `MIMO MLP (0.5690)`.

Финальный submission с этим пайплайном был отправлен на Kaggle и получил **Private Score 0.66**. Для выбора модели мы ориентировались именно на локальную temporal validation, но итоговый score подтверждает, что схема валидации была разумной и не расходилась с поведением на соревновании слишком сильно.

7.2 Feature importance

По встроенной оценке важности признаков у CatBoost на первом месте оказались: 1. **`item_nbr`** — товар задаёт базовый уровень спроса 2. **`store_nbr`** — магазины заметно отличаются по масштабу и структуре продаж 3. **`lag_7, lag_14`** — недельные лаги хорошо ловят локальную сезонность 4. **`sales_mean_7d`** — короткое окно помогает подхватывать текущий уровень спроса 5. **`day_of_week`** — недельный цикл остаётся одним из самых устойчивых паттернов 6. **`onpromotion`** — промо-акции действительно существенно сдвигают продажи

Для этой задачи это ожидаемый результат: сначала модель должна понять, **что** продаётся и **где**, а уже потом уточнять прогноз за счёт недавней динамики и календарных факторов.

7.3 Анализ ошибок

Чтобы понять, где модель ошибается сильнее всего, мы посмотрели разрезы по товарным семействам и типам магазинов.

Топ-5 самых сложных `product family` по NWRMSLE:

Product family	NWRMSLE	Число строк в val
SCHOOL AND OFFICE SUPPLIES	0.7668	2,470
BEVERAGES	0.5879	265,409
LIQUOR,WINE,BEER	0.5648	13,288

Product family	NWRMSLE	Число строк в val
POULTRY	0.5624	22,601
GROCERY II	0.5469	3,747

Здесь есть две разные ситуации. Категория **SCHOOL AND OFFICE SUPPLIES** формально самая тяжёлая, но у неё очень мало наблюдений на валидации, поэтому метрика там шумнее и чувствительнее к отдельным промахам. Гораздо интереснее **BEVERAGES**: ошибка тоже высокая (`0.5879`), но при этом это очень крупная категория с `265,409` строками. То есть проблема не в редкости данных, а в том, что спрос в этой группе действительно сложнее моделировать из-за сильной волатильности и влияния промо.

Отдельно выделяются **LIQUOR,WINE,BEER** и **POULTRY**. Для первой группы можно ожидать более резкие всплески около специальных дат и промо, а для второй добавляется чувствительность скоропортящихся товаров, которые в метрике штрафуются сильнее. **GROCERY II** тоже попадает в верхнюю часть списка скорее как более редкая и менее стабильная категория.

Ошибка по типам магазинов:

Store type	NWRMSLE
E	0.533
B	0.514
A	0.514
C	0.507
D	0.488

По типам магазинов разброс уже заметно меньше, чем по `family`: между лучшим и худшим типом разница составляет около `0.045` пункта NWRMSLE. Самым сложным оказался тип **E** (`0.533`), а самым стабильным, наоборот, тип **D** (`0.488`). Это важно, потому что первоначальная гипотеза про тип **D** как самый проблемный не подтвердилась после точного пересчёта.

В целом из анализа ошибок видно, что для CatBoost основная сложность лежит не столько на уровне формального типа магазина, сколько на уровне конкретных товарных категорий с нестабильным спросом, промо-эффектами и высокой дисперсией продаж.

---

8. Выводы

Что дало наибольший прирост

1. **Хорошие признаки**. Самый сильный вклад дали лаги и rolling-статистики на нескольких уровнях агрегации: `store × item`, `item`, `store`, `store × family`
2. **Аккуратная обработка данных**. Чтение чанками, sparse grid и нормальная очистка возвратов позволили вообще сделать задачу вычислимо устойчивой
3. **CatBoost с категориальными признаками**. Возможность напрямую использовать `store_nbr`, `item_nbr`, `family`, `type` оказалась очень полезной

Почему ансамбль не улучшил CatBoost

- NN обучалась на ограниченной подвыборке, поэтому просто уступала CatBoost по качеству
- Сам CatBoost уже достаточно хорошо снимал основные паттерны в данных, и ансамблю было трудно добавить что-то новое
- Оптимальный вес `alpha = 0.8` показывает, что даже при смешивании итоговый прогноз почти полностью опирался на CatBoost

Что можно улучшить дальше

- попробовать **LightGBM** как ещё одну сильную табличную модель
- дообучить NN на большем объёме данных и аккуратнее оптимизировать `DataLoader`
- перейти от простого blending к **stacking**
- сделать **per-horizon ensemble**, где вес модели зависит от дня прогноза
- добавить внешние признаки, например погодные или макроэкономические данные

Если формулировать главный итог коротко, то нас больше всего убедило следующее: на очень большом табличном датасете с большим числом рядов выиграл не самый сложный по форме подход, а наиболее аккуратно реализованный пайплайн с сильным feature engineering. Самой сложной частью проекта оказалась даже не настройка модели, а организация данных и памяти так, чтобы можно было быстро и стабильно прогонять эксперименты. Отдельно удивило, что нейросеть сама по себе не дала прироста: без сопоставимого качества базовых моделей ансамбль не спасает, а только слегка усредняет уже имеющийся результат.